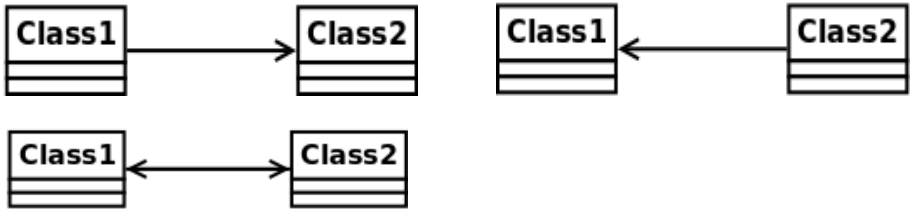
Java Persistance API (JPA2)
Beziehungen

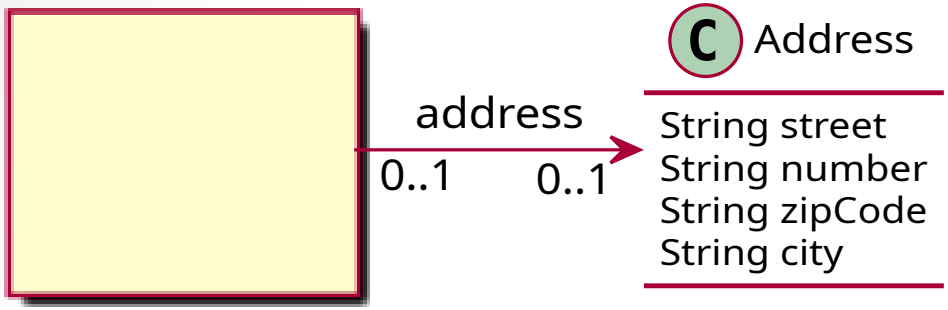
Beziehungen zw. Tabellen

- 3 Arten (Multiplizität / Kardinalität)
  - 1:1
  - 1:n
  - n:m
- Möglichkeiten zur Navigation im Klassenmodell
  - Unidirektional:
  - Bidirektional:



- Ausgestaltung im Datenmodell
  - Ohne assoziative Tabelle (nur bei 1:1 und 1:n möglich)
  - Mit assoziativer Tabelle (1:n, n:m)

1:1 Beziehung unidirektional




```java
@Entity
public class Employee {
    @Id
    @GeneratedValue
    Integer id;

    int salary;
    String firstName;
    String lastName;

    @OneToOne
    Address address;
}
```

```java
@Entity
public class Address {
    @Id
    @GeneratedValue
    Integer id;

    String street;
    @Column(name = "addrNum")
    String number;
    String zipCode
    String city;
}
```

1:1 Beziehung unidirektional

- Es müssen alle miteinander in Beziehung stehenden Entitäten im Zustand "managed" sein:

```
Employee emp = new Employee(800, "Hermann", "Mayr");
Address a1 = new Address("Himbeerstrasse", "22a", "4440", "Steyr");
emp.setAddress(a1);

em.persist(a1);
em.persist(emp);
```

- Standardmäßig wird bei unidirektionalen Beziehungen eine Assoziativtabelle angelegt / verwendet (da Address keine Foreign-Key-Spalte besitzt).

  Ist dies nicht erwünscht (meistens), kann JPA durch die Annotation @JoinColumn angewiesen werden, eine Foreign-Key-Spalte in der Tabelle der referenzierenden Entität anzulegen:

```
@JoinColumn(name = "addressid")
Address address;
```

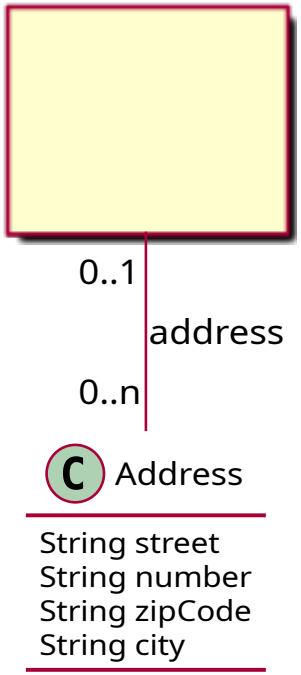
Cascade

- Transitive Persistenz (Persistence by Reachability) bedeutet, dass ein Entity nicht nur explizit persistent gemacht werden kann, sondern dass es implizit (automatisch) persistent wird, wenn es von einem bereits persistenten Entity referenziert wird.
- Mit dem Attribut *cascade* kann angegeben werden, dass eine Operation am einen Ende der Beziehung auch für die Entität(en) am anderen Ende wirksam wird.
- Beispiel: Wird *Employee* persistiert, so geschieht dies auch mit der referenzierten Entität *Address* automatisch:

```
@OneToOne(cascade = CascadeType.PERSIST)
Address address;


Employee emp = new Employee(800, "Hermann", "Mayr");
Address a1 = new Address("Himbeerstrasse", "22a", "4440", "Steyr");
emp.setAddress(a1);

//em.persist(a1); a1 wird von emp referenziert und somit automatisch persistiert
em.persist(emp);
```

1:1 Beziehung bidirektional

- Über das Attribut mappedBy das andere "Ende" einer bidirektionalen Beziehung festgelegt werden
- mappedBy bezieht sich dabei auf die Membervariable
- Eine Foreign-Key-Spalte wird automatisch angelegt, kann allerdings weiterhin mit @JoinColumn konfiguriert werden




```java
@Entity
public class Employee {
    @Id
    @GeneratedValue
    Integer id;

    int salary;
    String firstName;
    String lastName;

    @OneToOne
    Address address;
}
```

```java
@Entity
public class Address {
    @Id
    @GeneratedValue
    Integer id;

    String street;
    @Column(name = "addrNum")
    String number;
    String zipCode
    String city;

    @OneToOne(mappedBy = "address")
    Employee employee;
}
```

HTL LEONDING

@OneToOne

Weitere Parameter:

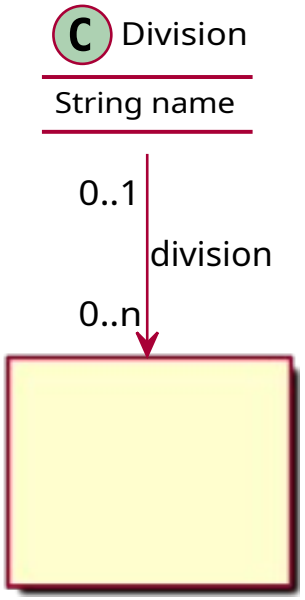
@OneToOne(...)			
Attribut	**Typ**	**Default**	**Beschreibung**
cascade	CascadeType[]	—	Zu kaskadierende Operationen auf der Zielseite der Assoziation
fetch	FetchType	EAGER	Ladezeitpunkt der Assoziation (EAGER, LAZY)
mappedBy	String	—	Name des Property der Beziehung auf der Eigentümerseite
optional	boolean	true	Angabe, ob Assoziation optional ist
orphanRemoval	boolean	false	Angabe, ob Orphan gelöscht werden soll
targetEntity	Class	void.class	Klasse des Ziels der Assoziation

1:1, 1:n - @JoinColumn

@JoinColumn(...)			
Attribut	**Typ**	**Default**	**Beschreibung**
column-Definition	String	—	Zusätzliches SQL-Fragment zur Definition der Spalte
insertable	boolean	true	Angabe, ob die Spalte in Insert-Anweisungen verwendet werden soll
name	String	—	Der Name der Fremdschlüsselspalte
nullable	boolean	true	Angabe, ob die Fremdschlüsselspalte Nullwerte zulässt
referenced-ColumnName	String	—	Name der Spalte, die durch diese Fremdschlüsselspalte referenziert wird
table	String	—	Name der Tabelle (nur sinnvoll bei Sekundärtabellen)
unique	boolean	false	Angabe, ob das Property ein eindeutiger Schlüssel ist
updatable	boolean	true	Angabe, ob die Spalte in Update-Anweisungen verwendet werden soll

1:n Beziehung unidirektional

- @OneToMany
- Das n-Ende wird über eine Collection (Set, List) abgebildet
- Wie bei @OneToOne wird standardmäßig eine Assoziativtabelle angelegt, dies kann durch Angabe einer @JoinColumn verhindert werden.
- Tipp: Collection im Kontruktor initialisieren

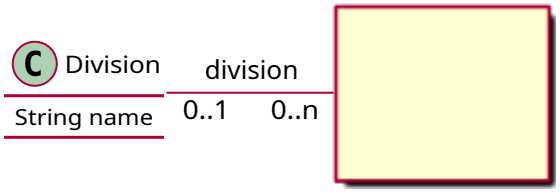



```
@Entity
public class Division {
    @Id
    @GeneratedValue
    Integer id;

    String name;

    @OneToMany
    List<Employee> employees;

    public Division() {
      employees = new ArrayList<>();
    }
```

HTL LEONDING

1:n Beziehung bidirektional

- @OneToMany auf Collection
- @ManyToOne am -1 Ende
- mappedBy auf @OneToMany




```java
@Entity
public class Division {
    @OneToMany(mappedBy = "div")
    List<Employee> employees;
}
```

```java
@Entity
public class Employee {
    @ManyToOne
    Division div;
}
```

mappedBy verbindet zwei unidirektionale Bezierungen zu einer bidirektionalen Beziehung.

mappedBy = "beide Enden gehören zusammen"

```java
Division div = new Division("div1");
Employee emp = new Employee(800, "Hermann", "Mayr");

emp.setDiv(div);
div.getEmployees().add(emp);

em.persist(div);
```

1:n Beziehung - Tipps

- Da bidirektionale 1:n-Beziehungen sehr häufig vorkommen, lohnt es sich, eine einfache Merkregel einzuprägen:
- Bei bidirektionalen 1:n-Beziehungen (oder n:1) ist die „n-Seite" die besitzende Seite
- Die Join-Spalte der besitzenden Seite wird optional mit @Joinnolunn annotiert
- Die „1-Seite" ist die inverse Seite, daher wird des mappedBy-Attribut verwendet

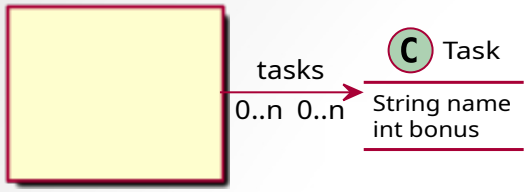
1:n - @OneToMany

@OneToMany(...)			
Attribut	**Typ**	**Default**	**Beschreibung**
cascade	CascadeType[]	—	Zu kaskadierende Operationen auf der Zielseite der Assoziation
fetch	FetchType	LAZY	Ladezeitpunkt der Assoziation (EAGER, LAZY)
mappedBy	String	—	Name des Property der Beziehung auf der Eigentümerseite. Bei bidirektionalen Beziehungen zwingend erforderlich
orphanRemoval	boolean	false	Angabe, ob Orphan gelöscht werden soll
targetEntity	Class	void.class	Zielklasse der Assoziation

1:n - @ManyToOne

@ManyToOne(...)			
Option	**Typ**	**Default**	**Beschreibung**
cascade	CascadeType[]	—	Zu kaskadierende Operationen auf der Zielseite der Assoziation
fetch	FetchType	EAGER	Ladezeitpunkt der Assoziation (EAGER, LAZY)
optional	boolean	true	Beziehung ist optional
targetEntity	Class	void.class	Klasse des Ziels der Assoziation

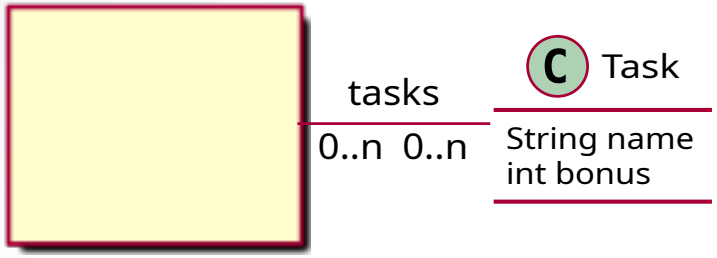
m:n Beziehung unidirektional




```java
@Entity
public class Employee {
    @Id
    @GeneratedValue
    Integer id;

    int salary;
    String firstName;
    String lastName;

    @ManyToMany
    List<Task> tasks;
}
```

```java
Employee emp = new Employee(800, "Hermann", "Mayr");
Task t = new Task();
emp.getTasks().add(t);
```

m:n Beziehung bidirektional




```java
@Entity
public class Employee {
    @Id
    @GeneratedValue
    Integer id;

    int salary;
    String firstName;
    String lastName;

    @ManyToMany
    List<Task> tasks;
}
```

```java
@Entity
public class Task {
    @Id
    @GeneratedValue
    int id;

    String name;
    int bonus;

    @ManyToMany(mappedBy = "tasks")
    List<Employee> employees;
}
```

```java
Employee emp = new Employee(800, "Hermann", "Mayr");
Task t = new Task();
emp.getTasks().add(t);
t.getEmployees().add(emp);
```

m:n - @ManyToMany

@ManyToMany(...)			
Option	**Typ**	**Default**	**Beschreibung**
cascade	CascadeType[]	—	Zu kaskadierende Operationen auf der Zielseite der Assoziation
fetch	FetchType	LAZY	Ladezeitpunkt der Assoziation (EAGER, LAZY)
mappedBy*	String	—	Name des Property der Beziehung auf der Eigentümerseite
targetEntity	Class	void.class	Zielklasse der Assoziation

m:n - @JoinColumn

@JoinTable(...)			
Attribute	**Typ**	**Default**	**Beschreibung**
catalog	String	—	Katalogname
inverse-JoinColumns	JoinColumn[]	{}	Fremdschlüsselspalte(n) der Join-Tabelle zur nichtbesitzenden Tabelle
joinColumns	JoinColumn[]	{}	Fremdschlüsselspalten der Join-Tabelle zur besitzenden Tabelle
name	String	—	Name der Join-Tabelle
schema	String	—	Schema-Name
unique-Constraints	unique-Constraint[]	{}	Unique-Constraints (Benutzung nur bei Generierung)